

Last modified on

Sports Data Soccer API - Remote Phases Of Play Feed (MA61)

Get details of each phase that happens in the game.

Overview

The Soccer **Remote Phases of Play** feed provides all Phases across both teams, as they occurred in chronological order. This feed will list each run that happens in the game. Each phase will have a start and stop time, team id, start frame, end frame (same frame id as MA49), phase label, multiple event references and multiple qualifiers. The data is based on events and in remote tracking.

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY
Updated on every stop in play during the game, including for the following types of events: goal, shot at goal, assist, free kick, offside given, cards, start of the half, end of the half, corner, substitution, lineup Note: There will always be a gap of at least 30 seconds between files unless a goal is scored	Post-match

Remote Phases Of Play (MA61)

USAGE	FEED
GET pass matrix data by fixture UUID (<code>fx</code> parameter - supports legacy IDs)	<code>/soccerdata/remotephasesofplay/{outletAuthk{queryParameters}}</code>

USAGE	FEED
GET pass matrix data by fixture UUID (path parameter)	/soccerdata/remotephasesofplay/{outletAuthKey}{queryParameters}

If your request URL is as follows (does not contain a fixture UUID), the feed returns an error:
/soccerdata/remotephasesofplay/{outletAuthKey}

Query Parameters And Example Request Parameters

This feed also makes use of the **Global query parameters**, and you can combine them with the following query parameters (some of which can be used in conjunction with each other. However, you must query the **/remotephasesofplay** feed by a specific fixture (match) UUID.

Legacy IDs: You can use the **fx** parameter to filter by legacy ID - only one ID is supported (you cannot pass multiple IDs in a comma-separated list).

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example **GET** requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You **MUST** format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the **main guide** and **API Overview and Authorisation guide**.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid **{outletAuthKey}** this must be after the Outlet Authentication Key (and endpoint, if applicable). for **_rt={mode}** (operating mode) and **_fmt={dataFormat}** (response format), entity query parameter and UUID.
- Begin the query parameter string with **?** (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the **&** (ampersand) operator. Values must be separated by the correct operator, for example **&** (multiple values, 'AND'), or - if a parameter supports it - **,** (multiple values, 'OR') or **!** ('NOT').
- Use the required **_** (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces **{}** from your calls (we include these as an example only).
- If using the JSONP format (**_fmt=jsonp**) you **MUST** use it in combination with the callback parameter (**_c1bk**) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>{fixtureUuid}</code> Fixture UUID	GET pass matrix data by the specified fixture UUID (path parameter method) . Pass the fixture UUID in the request URL directly as the <code>{assetId}</code> (asset/resource ID). GET https://api.performfeeds.com/soccerdata/remotephasesofplay/{outletAuthKey}/{fixtureUuid?_rt={mode}&_fmt={dataFormat}
<code>fx</code>	Get match pass matrix data by the specified fixture UUID (query parameter method) . Pass the fixture UUID to the <code>fx</code> parameter. GET https://api.performfeeds.com/soccerdata/remotephasesofplay/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&fx={fixtureUuid}

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML](#)
- [JSON](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

- ▶<remotePhasesOfPlay> (XML only)
- ▶<matchInfo>
- ▶<description>
- ▶<sport>
- ▶<ruleset>
- ▶<competition>
- ▶<country>
- ▶<tournamentCalendar>
- ▶<stage>
- ▶<series>
- ▶<contestants> (XML only)
- ▶<contestant>
- ▶<country>

- ▶<venue>
- ▶<liveData>
- ▶<matchDetails>
- ▶<periods> (XML only)
- ▶<period>
- ▶<suspensions>
- ▶<suspension>
- ▶<scores>
- ▶<ht>
- ▶<ft>
- ▶<et>
- ▶<pen>
- ▶<total>
- ▶<aggregate>
- ▶<totalUnconfirmed>
- ▶<goal>
- ▶<missedPenalties>
- ▶<card>
- ▶<substitute>
- ▶<VAR>
- ▶<penaltyShootout>
- ▶<penaltyShot>
- ▶<lineUp>
- ▶<player>
- ▶<teamOfficial>
- ▶<matchDetailsExtra>
- ▶<attendance>
- ▶<awayAttendance>

▶<matchOfficials> (XML only)

▶<matchOfficial>

▶<phaseByPhase>

▶<phase>

▶<phaseLabel>

▶<expectedThreat>

▶<directPassingSummary>

▶<directPassing>

▶<fastPassingTempoSummary>

▶<directPassing>

▶<overloadSummary>

▶<directPassing>

▶<compactnessSummary>

▶<compactness>

▶<phaseSummary>

▶<stat>

Definitions Repeated Throughout The Feed

There are some definitions repeated throughout the feed and are defined here.

- **id** (int): a unique identifier (unique per Phase per game)
- **startTime** (float): the start time of the Phase within the current period in seconds
- **endTime** (float): the end time of the Phase within the current period in seconds
- **startFrame** (int): the start frame of the Phase as seen in the Opta Vision in-venue or remote feeds.
- **endFrame** (int): the end frame of the Phase as seen in the Opta Vision in-venue or remote feeds
- **startX** (float): the x-coordinate in the first frame of the Phase
- **startY** (float): the y-coordinate in the first frame of the Phase
- **endX** (float): the x-coordinate in the final frame of the Phase
- **endY** (float): the y-coordinate in the final frame of the Phase

Feed Definitions

Below are links to, or brief explanations of, entries in the MA60

- **start**

- `startTime/startFrame`
- `startX/startY`
- `firstTouchPlayerId, firstTouchEventId` (int): the event_id of the first Opta Touch Event during the phase, and the corresponding player_id for this touch event. Note that not all phases have to include a touch event/player.
- `initiatorPlayerId, initiatorEventId` (int): Defined in detail here. Note that not all phases have to include an initiator player/event.
- `end`
 - `endTime/endFrame`
 - `endX/endY`
- `phase_label`
 - `value` (str): the base phase as defined here
- `directPassingSummary` : defined under Direct Passing Play here
 - `directPassing`
 - `id`
 - `startTime/startFrame`
 - `endTime/endFrame`
 - `relatedPassEvents` (array of ints): Opta Pass Event ID's which occurred during this particular Direct Passing timeframe.
- `fastPassingTempoSummary`: defined under Fast Passing Tempo here
 - `fastPassingTempo`
 - `id`
 - `startTime/startFrame`
 - `endTime/endFrame`
 - `relatedPassEvents` (array of ints): Opta Pass Event ID's which occurred during this particular Fast Passing Tempo timeframe.
- `overloadSummary` : defined under Overload here
 - `overload`
 - `id`
 - `startTime/startFrame`
 - `endTime/endFrame`
 - `attackerCount` (int): the maximum number of attackers involved in the overload
 - `defenderCount` (int): whilst we have the maximum number of attackers during the overload, what was the minimum number of defenders involved in the overload
 - `overloadValue` (int): what was the largest attacker count advantage during the overload
 - `overloadType` (str): one of either `wide` or `central` labels based on **Attacking Wide Overload** and **Central Overload** defined under Overload here
 - `averageBallX` (float): the average x-coordinate of the ball during the overload
 - `averageBallY` (float): the average y-coordinate of the ball during the overload
 - `attackingPlayerIds` (list - str): the Player UUID's for all players on the in-possession team who were involved with the overload at the maximal moment in the overload (i.e. when the moment which the `overloadValue`, `attackerCount` and `overloadValue` were calculated). The number of players in this list should match the number in `attackerCount`.
 - `attackingPlayerIds` (list - str): the Player UUID's for all players on the in-possession team who were involved with the overload **at any moment**. This could include more players than

attackingPlayerIds since some players might have only been involved in the overload before or after it's maximal point.

- **defendingPlayerIds** (list - str): the Player UUID's for all players on the out-of-possession team who were involved with the overload at the maximal moment in the overload (i.e. when the moment which the **overloadValue**, **attackerCount** and **overloadValue** were calculated). The number of players in this list should match the number in **defenderCount**.
- **defendingPlayerIds** (list - str): the Player UUID's for all players on the out-of-possession team who were involved with the overload **at any moment**. This could include more players than **defendingPlayerIds** since some players might have only been involved in the overload before or after it's maximal point.
- **defensiveCompactnessSummary** : defined under Defensive Compactness here
 - **defensiveCompactness**
 - **id**
 - **startTime/startFrame**
 - **endTime/ endFrame**
 - **label** (str): one of either **high/medium/low** as defined under Defensive Compactness here
 - **averageDefensiveAreaCoverage** (float): average Convex Hull area covered covered by the defensive team during the phase (in meters squared)
- **phaseSummary** : This node contains various aggregates across the entirety of the Phase.
 - **numberPasses** (int): number of pass events during this Phase
 - **distanceProgressedUpField** (float): distance (in metres) between the first and last x-coordinate during the phase. Can be negative if the ball ends the phase at a smaller x-coordinate. Note: this does not take into account where the ball may have gone during the middle of the phase
 - **distanceTravelledByBall** (float): distance (in meters) travelled by the ball during the phase. This ignores z-coordinates (e.g. height) and is calculated as the summed distance over every frame at 25 frames-per-second during the phase.
 - **phaseDuration** (float): time (in seconds) that this Phase lasts.
 - **includesGoal** (bool): whether or not this Phase includes a goal.
 - **includesShot** (bool): whether or not this Phase includes a shot.
 - **averageDefendingTeamHorizontalWidth/averageAttackingTeamHorizontalWidth** (float): average width (in metres) of defending/attacking team (maximum y-coordinate difference between defending/attacking team players for each frame)
 - **averageDefendingTeamVerticalLength/averageAttackingTeamVerticalLength** (float): average length (in metres) of defending/attacking team (maximum x-coordinate difference between defending/attacking team players for each frame)
 - **averageDefendingTeamHeightLastDefender/averageAttackingTeamHeightLastDefender** (float): average minimum y-coordinate of outfield players in the defending/attacking team
 - **mostCommonDefensiveCompactness** (str): most common label value from the above **defensiveCompactnessSummary** node (in terms of total seconds in each label value)
 - **numberHighPressureOnReceiver** (int): number of high pressures applied by the defending team on attacking team receptions. Note: we only count high pressures applied to the attacking team labelled as the possessionTeamId of the given Phase. For example, if there is a brief period of disputed possession during the Phase, we won't count high pressures on the team not labelled as the possessionTeamId of the Phase.
 - **numberHighPressureOnTouches** (int): number of high pressures applied by the defending team on attacking team Opta Touch events. Note: we only count high pressures applied to the attacking team labelled as the possessionTeamId of the given Phase. For example, if there is a brief period of disputed

possession during the Phase, we won't count high pressures on the team not labelled as the possessionTeamId of the Phase.

- **numberAttackingPlayersWithTouches** (int): number of unique players on the attacking team that have Opta Touch events during the phase
- **numberRuns** (int): total number of off-ball Runs from the attacking team.
- **numberDangerousRuns** (int): total number of off-ball Runs from the attacking team which are labelled as Dangerous (dangerous label for Runs defined here)
- **numberLineBreakingActions** (int): number of Line Breaking Passes and Line Breaking Carries.